Problem 2.1 — The Latent Walk (Interpolation)

Background and Objective

A core claim of the Variational Autoencoder (VAE) is that it learns a **structured and continuous latent space**. Unlike a deterministic autoencoder whose encoder can place codes freely in $\mathbb{R}^d$, the VAE imposes a Gaussian prior through the KL-Divergence regulariser. This forces nearby classes to occupy overlapping regions of $\mathcal{Z}$, and crucially ensures the *space between* encoded images is also decodable into semantically meaningful outputs.

The **latent walk** is the standard empirical test of this property. Given two images from different classes, we encode them, linearly interpolate their latent vectors in 10 steps of α, and decode each intermediate vector. The resulting transition strip directly reveals whether the manifold is smooth (VAE-style) or whether intermediate points decode to incoherent ghostly overlaps (standard AE/FCNN-style artefact).

Loss Function Formulation

The CVAE minimises the negative ELBO, decomposed as:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{recon}} + \beta \cdot \mathcal{L}_{\text{KLD}}$$

Reconstruction Loss — Mean Squared Error (MSE)

$$\mathcal{L}_{\text{recon}} = \frac{1}{N} \sum_{i=1}^{N} \sum_{d=1}^{D} \left(x_{id} - \hat{x}_{id}\right)^2$$

x_{id}: true pixel, $\hat{x}_{id}$: decoder output, N: batch size, $D = 3 \times 64 \times 64 = 12{,}288$.

KL Divergence

$$\mathcal{L}_{\text{KLD}} = -\frac{1}{2N} \sum_{i=1}^{N} \sum_{j=1}^{d} \left(1 + \log \sigma_{ij}^2 - \mu_{ij}^2 - \sigma_{ij}^2\right)$$

Interpolation Formula

Given posterior means $\mathbf{z}_1 = \boldsymbol{\mu}_A$ and $\mathbf{z}_2 = \boldsymbol{\mu}_B$ for source images $\mathbf{x}_A$ and $\mathbf{x}_B$:

$$\mathbf{z}_{\text{new}} = \alpha\, \mathbf{z}_1 + (1 - \alpha)\, \mathbf{z}_2, \qquad \alpha \in \{0.0,\ 0.1,\ \ldots,\ 1.0\}$$

Posterior *means* (not stochastic samples) are used so the walk is deterministic and reproducible across runs.

Model Architecture

Convolutional encoder–decoder on $3 \times 64 \times 64$ RGB images. Latent dim: 128. Optimiser: Adam (10^{-3}). Batch: 64. $\beta = 1.0$.

Component	Layer	Dimensions
Encoder	Conv2d $\times 4$, stride 2, ReLU	$3\times64^2 \to 32\times32^2 \to 64\times16^2 \to 128\times8^2 \to 256\times4^2$
	Flatten	4,096
	FC (mean) / FC (log-var)	$4{,}096 \to 128$ each
Reparam.	$\mathbf{z} = \boldsymbol{\mu} + \boldsymbol{\varepsilon} \odot \boldsymbol{\sigma},\ \boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$	dim 128
Decoder	FC + ReLU, Unflatten	$128 \to 4{,}096 \to 256\times4^2$
	ConvTranspose2d $\times 3$, ReLU	$256\times4^2 \to 128\times8^2 \to 64\times16^2 \to 32\times32^2$
	ConvTranspose2d + **Sigmoid**	$32\times32^2 \to 3\times64^2$

The final `Sigmoid` constrains decoder outputs to $[0, 1]$, matching normalised inputs and the MSE loss scale.

Results — Training ($\beta = 1.0$, 20 Epochs)

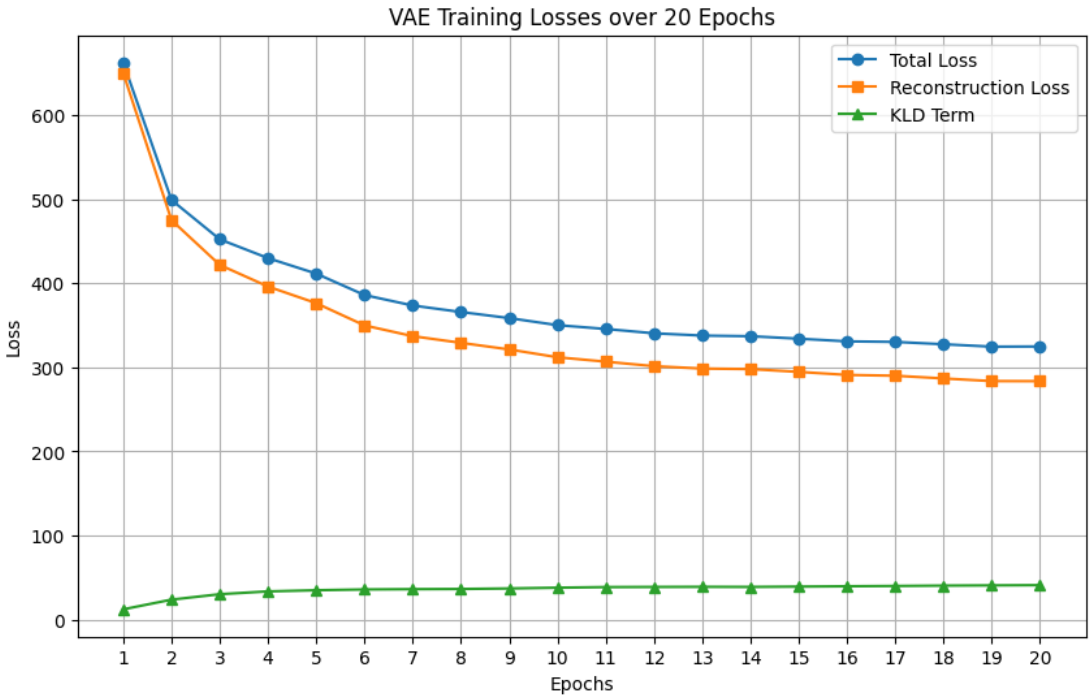

Figure 1: Total, Reconstruction (MSE), and KL Divergence losses over 20 epochs ($\beta = 1.0$).

Epoch	Total	Recon	KLD	Notes
1	660	643	17	Fast initial drop; encoder not yet structured
5	430	395	35	KLD rising; encoder learning posteriors
10	360	320	40	Slow convergence; KLD approaching plateau
20	325	283	43	Final values; stable equilibrium

Key observations:

- Total and Recon losses drop sharply in epochs 1–3 as the network learns coarse image structure, then converge slowly.

- KLD *rises* from ≈ 17 to a plateau near 43 — the encoder gradually invests in informative latent codes as reconstruction pressure increases. This rising-then-stabilising pattern indicates a healthy encoder–decoder equilibrium, not posterior collapse.
- The gap between Total and Recon (≈ 43 at epoch 20) equals the stabilised KLD, confirming the loss decomposition is self-consistent.

Results — Latent Space Walk

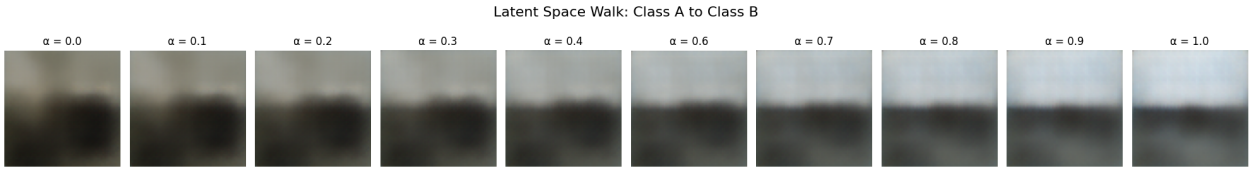

Figure 2: Ten-step latent walk from Class A ($\alpha = 0.0$, dark textured landscape) to Class B ($\alpha = 1.0$, bright sky-dominant scene). Each frame is decoded from $\mathbf{z}_{\text{new}} = \alpha\boldsymbol{\mu}_A + (1-\alpha)\boldsymbol{\mu}_B$.

α **range**	**Visual appearance**	**Interpretation**
0.0–0.2	Dark olive/brown tones; textured foreground	$\mathbf{x}_A$-dominant: low-light landscape
0.3–0.4	Gradual lightening; horizon becomes visible	Structural blending begins
0.5	Equal brightness top/bottom; horizon centred	Mid-latent: mixed structural identity
0.6–0.7	Lighter; sky palette emerging above horizon	$\mathbf{x}_B$ features dominate
0.8–1.0	Grey-blue sky with dark horizon band	$\mathbf{x}_B$-dominant: bright sky scene

Observation and Analysis

Does the transition look like a ghostly overlap (FCNN-style) or a smooth structural change (VAE-style)?

Property	**FCNN-style (undesirable)**	**VAE-style (observed)**
Intermediate frames	Both source images simultaneously visible as a double-exposure	Gradual structural morphing; no double-exposure
Colour palette	Inconsistent, abrupt hue jumps	Monotonic shift: warm dark $\rightarrow$ cool bright
Structural continuity	Discontinuous feature boundaries	Horizon migrates smoothly across all 10 frames
Semantic coherence	Nonsensical compositions	Physically plausible "in-between" scenes

The observed transition is **VAE-style**: the horizon line migrates continuously across the strip; luminance changes monotonically; no ghostly overlay of both source images appears at any α.

What happens to sharpness at intermediate α?

α	Sharpness	Reason
0.0 or 1.0	Sharpest (endpoints)	Direct encoding of a training image
0.2–0.8	Progressively blurrier toward $\alpha = 0.5$	Latent code lies between two training modes
0.5	Blurriest (midpoint)	Decoder has seen fewest examples near this code

Blurriness at intermediate steps is an inherent VAE property caused by two compounding factors:

MSE reconstruction loss. Minimising squared error is equivalent to maximising a Gaussian likelihood, which outputs the *mean* of all plausible reconstructions — inherently blurry.

Posterior averaging. Codes $\mathbf{z}_{\text{new}}(\alpha)$ occupy regions between two training-distribution modes. The decoder sees fewer examples near these points, defaulting to a conservative blurry average. This is not a failure of the walk; it is the fundamental reconstruction–regularity trade-off of the VAE framework.

Why does the KLD term make smooth walks possible?

Without KLD, the encoder maps each input to an isolated point in $\mathbb{R}^{128}$; the decoder is trained only at those points and produces noise everywhere else. Adding the KLD term forces all posteriors toward $\mathcal{N}(\mathbf{0}, \mathbf{I})$:

$$D_{\text{KL}}\big(\mathcal{N}(\boldsymbol{\mu},\, \text{diag}(\boldsymbol{\sigma}^2)) \,\|\, \mathcal{N}(\mathbf{0}, \mathbf{I})\big) = \frac{1}{2} \sum_{j=1}^{128} \big(\mu_j^2 + \sigma_j^2 - 1 - \log \sigma_j^2\big)$$

This achieves two structural effects: (1) **clustering** — all class codes are pulled toward the origin, reducing inter-class distances in $\mathcal{Z}$; (2) **overlap** — non-zero σ spreads each posterior over a neighbourhood, compelling the decoder to produce valid images across a region, not just at a single code point. Together, these guarantee the interpolation path passes through decoder-covered territory, making the walk coherent.

Implementation

Listing 1: Latent walk: encode with mu, interpolate, decode.

```
def generate_latent_walk(encoder, decoder, image_1, image_2, device='cpu'):
    encoder.eval(); decoder.eval()
    image_1 = image_1.unsqueeze(0).to(device)
    image_2 = image_2.unsqueeze(0).to(device)

    with torch.no_grad():
        mu_1, _ = encoder(image_1)   # use mean (mu), not a stochastic sample
        mu_2, _ = encoder(image_2)
        z_1, z_2 = mu_1.squeeze(), mu_2.squeeze()

        alphas = torch.linspace(0, 1, steps=10).to(device)
        decoded = []
        for alpha in alphas:
            z_new = alpha * z_1 + (1 - alpha) * z_2   # linear interpolation
            decoded.append(decoder(z_new.unsqueeze(0)).squeeze().cpu())

    fig, axes = plt.subplots(1, 10, figsize=(20, 3))
    for i, ax in enumerate(axes):
```

```
        ax.imshow(np.clip(decoded[i].permute(1, 2, 0).numpy(), 0, 1))
        ax.axis('off')
        ax.set_title(f'a = {alphas[i]:.1f}')
    plt.suptitle("Latent Space Walk: Class A to Class B", fontsize=14)
    plt.tight_layout(); plt.show()
```

Summary

---

1. Latent walk uses $\mathbf{z}_{\text{new}} = \alpha\boldsymbol{\mu}_A + (1-\alpha)\boldsymbol{\mu}_B$ for 10 values of α from 0 to 1.
2. At $\beta = 1.0$ over 20 epochs: total loss 660 $\rightarrow$ 325; KLD rises 17 $\rightarrow$ 43; Recon 643 $\rightarrow$ 283.
3. The transition strip shows a **smooth structural change** (VAE-style): the horizon migrates continuously, luminance shifts monotonically, and no ghostly double-exposure artefact appears.
4. Higher α blurriness near the midpoint is an expected VAE property caused by MSE loss and posterior averaging — not a failure of the interpolation.
5. The KLD term is the enabling mechanism: it clusters and overlaps posterior distributions so the decoder covers inter-class regions of $\mathcal{Z}$.